

Name of the Student	S. Yuva Guru Mani Varma
Reg. No	192425005
GitHub Link	https://github.com/YuvaGuruManiVarmaS-018

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04

SLOT :

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

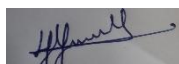
Duration: 1 Hour

Assessment Tool Weightage: 35%

Marks: 50

Code of Conduct:

I S. Yuva Guru Mani Varma/ 192425005 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: S. Yuva Guru Mani Varma

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

Signature of the Faculty:

Total Marks :

QUESTIONS:01

Total Marks: 50

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression

Round 1:

Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

Round 2:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	CNN Layer Matcher	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	Filter Finder	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	Parameter Sharing Puzzle	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	CNN Architecture Debugger	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	AlexNet vs ResNet Challenge	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	Edge Detection Explorer	Apply simple (3\times3) filters to an image and determine how convolution operations can detect edges and other basic features.
Level 2	Feature Map Detective	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.

Level	Challenge	Description
Level 3	Regularization Rescuer	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	CNN Architecture Race	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	CNN Application Master	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

Level 1 – CNN Layer Matcher

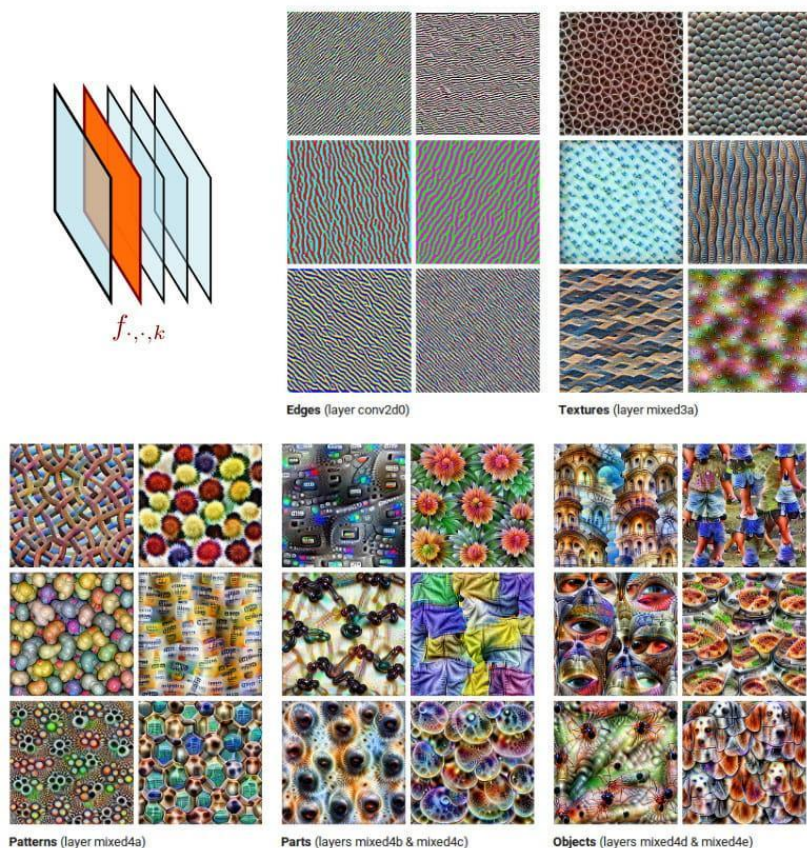
CNN Layer Function

Convolution Layer	Applies filters/kernels to the input image to extract features such as edges, textures, and shapes.
Pooling Layer	Reduces the spatial dimensions of feature maps while retaining important information.
Flatten Layer	Converts the 2D/3D feature maps into a 1D vector.
Fully Connected Layer	Uses the extracted features to perform classification and produce the final output.

Answer:

The normal CNN flow is:

Input Image → Convolution → ReLU → Pooling → Convolution → Pooling → Flatten → Fully Connected → Output



Level 2 – Filter Finder

Filters are small matrices, such as 3×3 , that slide over an image to detect features.

Example edge-detection filter:

This filter can detect **horizontal edges**.

- **Filter size:** Determines the local area examined.
- **Stride:** Determines how many pixels the filter moves at each step.
- **Padding:** Adds pixels around the image to control the output size.

For example, with a 5×5 image, 3×3 filter, stride = 1, padding = 0:

So the output feature map is 3×3 .

Level 3 – Parameter Sharing Puzzle

In CNNs, the **same filter weights are reused at different locations** of an image. This is called **parameter sharing**.

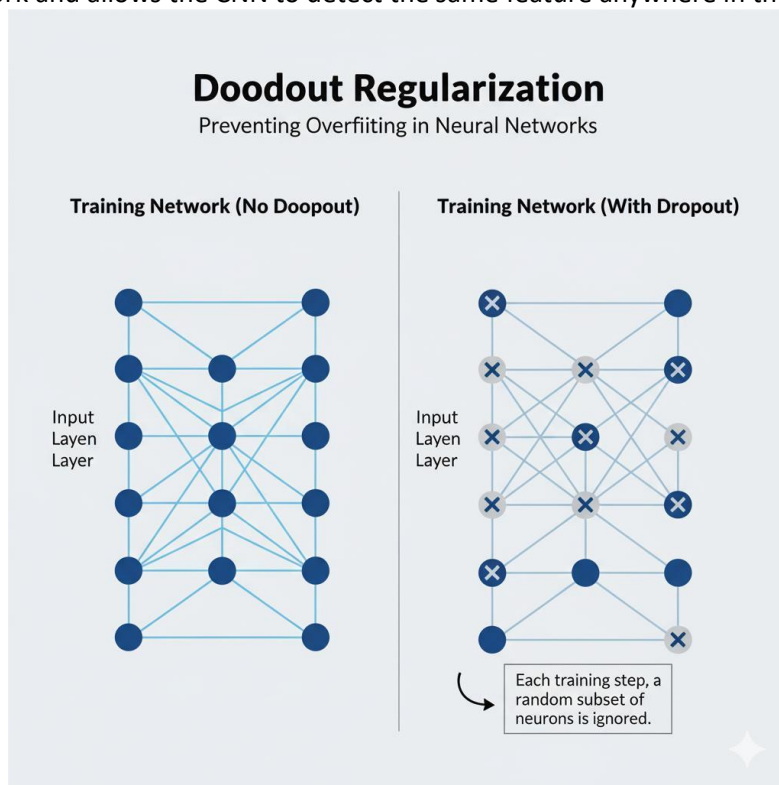
For a convolution layer with:

- Filter size = 3×3
- Input channels = 3
- Number of filters = 32

Number of weights:

Including one bias for each filter:

Answer: Parameter sharing greatly reduces the number of parameters compared with a fully connected network and allows the CNN to detect the same feature anywhere in the image.



Level 4 – CNN Architecture Debugger

A correct CNN architecture should maintain compatible dimensions between layers.

Incorrect example:

Input → Fully Connected → Convolution → Pooling

This is incorrect because convolution is normally applied to image/feature-map data before flattening.

Corrected architecture:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Important checks:

1. Convolution should operate on image/feature-map data.

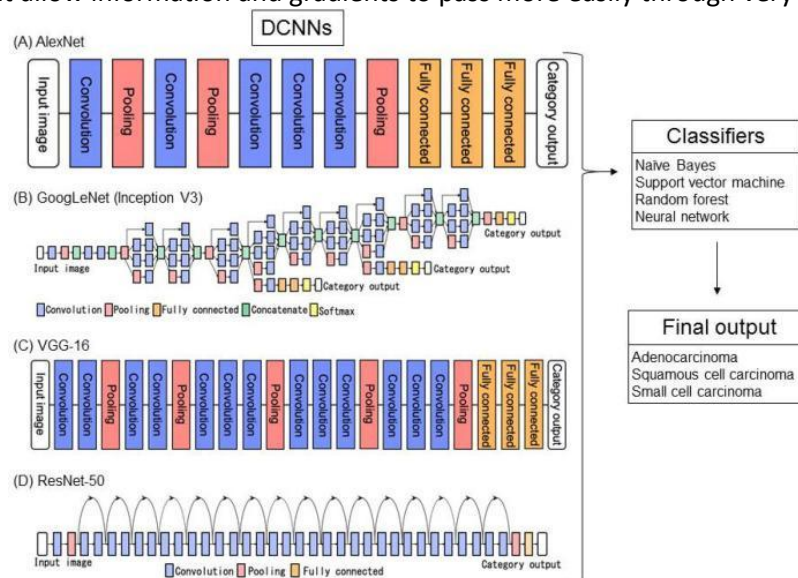
2. Pooling should follow suitable convolution/activation stages.
3. **Flatten** should normally occur before fully connected layers.
4. The dimensions entering each layer must be compatible.
5. The final fully connected layer should have the required number of output classes.

Level 5 – AlexNet vs ResNet Challenge

Feature	AlexNet	ResNet
Architecture	Deep CNN	Very deep CNN
Main idea	Large convolutional architecture	Residual/skip connections
Key feature	ReLU, pooling, dropout	Residual blocks
Problem addressed	Large-scale image classification	Difficulty training very deep networks
Skip connections	No	Yes
Depth	8 learned layers	Can have 18, 34, 50, 101, 152 layers
Advantage	Historically important and effective	Better suited for very deep networks

Answer:

AlexNet uses a conventional sequential CNN structure, whereas ResNet introduces **residual connections** that allow information and gradients to pass more easily through very deep networks.



ROUND 2 – CNN Feature Detective

Level 1 – Edge Detection Explorer

A **3×3 convolution filter** can detect edges by responding strongly to changes in pixel intensity.

Example vertical-edge filter:

When this filter slides across an image:

- Similar neighboring pixels → small response.
- Sudden intensity change → large response.
- Large response → likely edge.

Answer:

Convolution detects basic image features by calculating the weighted sum of neighboring pixels using a filter.

Level 2 – Feature Map Detective

CNNs learn features progressively.

Early layers:

- Edges
- Lines
- Corners

- Simple textures

Middle layers:

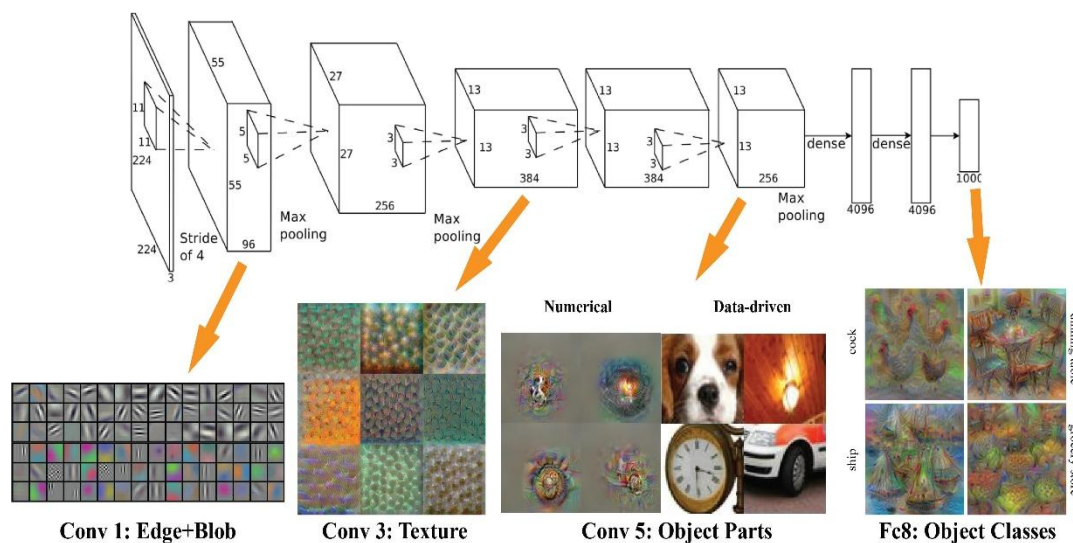
- Shapes
- Patterns
- Object parts

Deep layers:

- Faces
- Animals
- Vehicles
- Complex objects

Answer:

As depth increases, CNN feature maps generally move from **simple low-level features to complex high-level representations**. This hierarchical feature extraction is one of the main reasons CNNs are effective for image recognition.



Level 3 – Regularization Rescuer

Problem:

If training accuracy is very high but validation/test accuracy is much lower, the CNN is likely suffering from **overfitting**.

Suitable solutions include:

1. **Dropout** – randomly disables some neurons during training.
2. **Data augmentation** – creates modified training images through transformations such as rotation, flipping, and cropping.
3. **L2 regularization** – penalizes excessively large weights.
4. **Early stopping** – stops training when validation performance stops improving.
5. **More training data** – helps the model generalize better.

Answer:

The best approach is to use regularization techniques such as **dropout and data augmentation** to reduce overfitting and improve generalization.

Level 4 – CNN Architecture Race

If the application requires a very deep network:

Choose ResNet.

Reason: ResNet uses **residual/skip connections**, which make training deep networks easier.

If the application requires understanding a historically important CNN:

Choose AlexNet.

Reason: AlexNet demonstrated the effectiveness of deep CNNs for large-scale image classification.

Overall answer:

For most modern image-classification applications between these two choices, **ResNet is generally more suitable**, especially when deeper architectures and strong feature learning are required.

Level 5 – CNN Application Master

Application: Medical X-ray Image Classification

Problem:

Classify X-ray images into categories such as normal and abnormal.

Proposed CNN solution:

Input X-ray → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Residual Blocks → Flatten/Global Pooling → Fully Connected → Classification

Why CNN?

CNNs can automatically learn important visual patterns such as:

- Edges
- Textures
- Shapes
- Abnormal regions

Why ResNet?

ResNet is suitable because its residual connections help train deeper networks and allow effective extraction of complex image features.

Output:

For a binary problem:

- **Normal**
- **Abnormal**

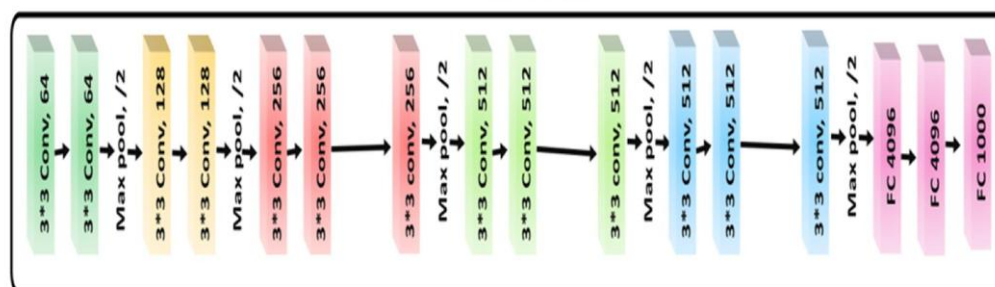
For multiple diseases, the final layer can contain multiple classes.

Final justification:

A CNN-based medical image classifier can automatically extract hierarchical visual features and classify X-ray images. ResNet is a strong architecture choice because residual connections facilitate deep feature learning and improve training of deeper networks.

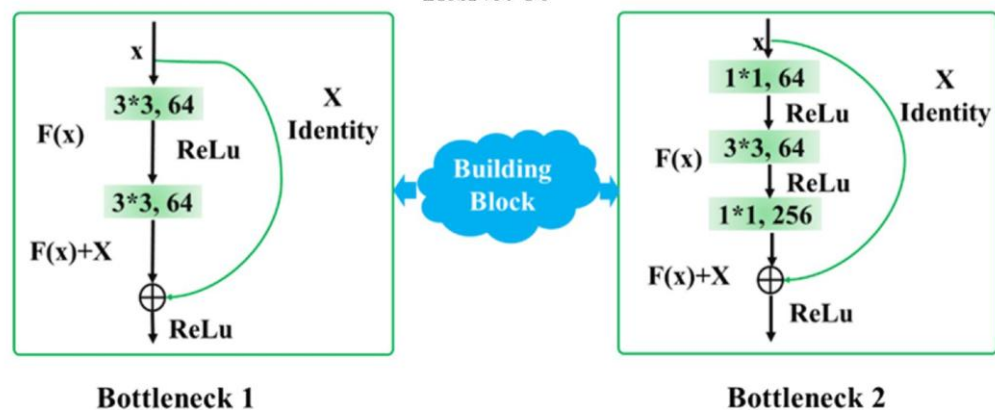
A

VGGNet-16



B

ResNet-50



Presentation Photo:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

